



Trabajo de Fin de Grado

Framework Backend–Frontend para el desarrollo rápido de proyectos apoyado en un modelo de datos en evolución: aplicación a los proyectos con el Jardín Botánico Viera y Clavijo

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Nicolás Rey Alonso

TUTORIZADO POR:
Rafael Juan Nebot Medina
María Dolores Afonso Suárez

Febrero de 2026

Resumen

El desarrollo de aplicaciones web de gestión de datos en entornos de investigación se enfrenta a una tensión recurrente: la necesidad de entregar software de calidad en plazos cortos frente a la inevitable evolución del modelo de datos a lo largo del proyecto. Cada vez que el modelo cambia, el coste de mantener sincronizados el backend, la capa de comunicación y la interfaz de usuario crece de forma desproporcionada, ya que buena parte de ese código se escribe a mano y se acopla a un dominio concreto.

Este Trabajo de Fin de Grado aborda dicho problema mediante el diseño, refactorización y consolidación de un *framework full-stack* reutilizable, compuesto por un backend en Python (*uniback*, basado en FastAPI, SQLAlchemy 2.x y PostgreSQL) y una librería de componentes Angular (*ngt-gui*). El núcleo de la contribución es una cadena de generación automática que parte del modelo ORM, deriva un *JSON Schema* enriquecido con pistas de presentación y, a partir de él, construye formularios dinámicos con Formly sin escribir HTML específico de cada entidad. El trabajo se estructura en una metodología iterativa incremental y se valida sobre las necesidades reales de los proyectos del Instituto Tecnológico de Canarias con el Jardín Botánico Viera y Clavijo.

Los resultados muestran un contrato de comunicación uniforme entre backend y frontend, un sistema de descubrimiento de entidades, una capa cliente genérica tipada y un generador de formularios guiado por el esquema, todo ello respaldado por pruebas automatizadas. El framework reduce drásticamente el código repetitivo necesario para exponer y editar una entidad nueva, y queda preparado para su reutilización en proyectos con modelos de datos en evolución.

Palabras clave: framework, generación automática de formularios, JSON Schema, FastAPI, Angular, Formly, modelo de datos en evolución, desarrollo rápido de aplicaciones.

Abstract

Building data-management web applications in research environments faces a recurring tension: delivering quality software on short schedules versus the inevitable evolution of the data model throughout the project. Every time the model changes, the cost of keeping the backend, the communication layer and the user interface in sync grows disproportionately, because much of that code is hand-written and coupled to a specific domain.

This Bachelor's Thesis tackles that problem through the design, refactoring and consolidation of a reusable full-stack framework, composed of a Python backend (*uniback*, based on FastAPI, SQLAlchemy 2.x and PostgreSQL) and an Angular component library (*ngt-gui*). The core contribution is an automatic generation pipeline that starts from the ORM model, derives an enriched JSON Schema with presentation hints and, from it, builds dynamic Formly forms without writing entity-specific HTML. The work follows an iterative and incremental methodology and is validated against the real needs of the projects carried out by the Instituto Tecnológico de Canarias with the Jardín Botánico Viera y Clavijo.

The results include a uniform communication contract between backend and frontend, an entity discovery system, a generic typed client layer and a schema-driven form generator, all backed by automated tests. The framework sharply reduces the boilerplate required to expose and edit a new entity, and is prepared for reuse in projects with evolving data models.

Keywords: framework, automatic form generation, JSON Schema, FastAPI, Angular, Formly, evolving data model, rapid application development.

Al ITC por concederme la oportunidad de participar en este proyecto

Índice general

Listings

4.1. Interfaz base de un plugin de esquema.	11
4.2. Ejemplo de objeto de pantalla dinámica (screen definition).	13
4.3. Payload del endpoint /bulk.	13
4.4. Respuesta del endpoint /bulk.	14
5.1. Función principal de generación de esquemas.	16
5.2. Registro de una entidad con la fábrica CRUD.	16
5.3. Lógica de clasificación de operaciones bulk.	16
5.4. Uso de savepoints para el modo partial.	17
5.5. Generación de una pantalla de tipo editable_table.	17
5.6. Fragmento del mapeo JSON Schema → Formly.	18
5.7. Mapeo de tipo a editor AG Grid.	18
5.8. Selección de vista en dynamic-page.component.html (fragmento). . . .	19
5.9. Fragmento del docker-compose.yml de desarrollo.	20
1. Estructura de ResponseEnvelope.	34

Índice de figuras

Índice de tablas

Capítulo 1

Introducción

“Los buenos programadores saben qué escribir. Los grandes saben qué reutilizar.”Eric S. Raymond [?]

1.1. Contexto

En la actualidad el software de gestión está sometido a cuatro presiones simultáneas: bajo coste, alta seguridad, alta eficiencia y un tiempo de desarrollo cada vez más reducido. El ritmo de evolución de las tecnologías y de los requisitos de negocio exige la capacidad de generar software de alta calidad en muy poco tiempo. Esto supone, en muchas ocasiones, una antítesis: para refinar y [testear](#) correctamente un producto es necesaria una inversión de tiempo que el calendario rara vez concede.

Para mitigar esta tensión la industria ha creado los [Frameworks](#). Estos productos de software empaquetan las funcionalidades transversales necesarias para el desarrollo rápido (persistencia, seguridad, comunicación, validación) permitiendo asegurar y probar la base del código y obtener así productos firmes, de bajo coste y alta calidad. Sin embargo, a pesar de tratarse de software muy maduro, los frameworks comerciales de propósito general tienen límites: es imposible crear una solución universal que satisfaga las necesidades individuales de cada desarrollo, por lo que siempre es necesaria la introducción de arquitecturas y convenciones propias que, apoyándose en la herramienta, completen los requisitos concretos del proyecto.

Es en ese punto donde se sitúa el presente Trabajo de Fin de Grado: en el desarrollo de las herramientas y funcionalidades añadidas que completan un framework propio, ajustado a las arquitecturas y a la forma de trabajar del [Instituto Tecnológico de Canarias \(ITC\)](#) en sus proyectos de gestión de datos científicos.

1.2. Descripción del problema

El [ITC](#), en colaboración con el [Jardín Botánico Viera y Clavijo \(JBVC\)](#), desarrolla aplicaciones web para la gestión de datos botánicos y de biodiversidad. Estos proyectos comparten una característica problemática: su modelo de datos no es estable, sino que *evoluciona* a medida que avanza la investigación, cambian los estándares y se incorporan nuevas fuentes y necesidades. Cada cambio en el modelo de datos propaga un coste en cascada que afecta al backend (modelos, validación, endpoints), a la capa de comunicación (contratos, tipos) y al frontend (formularios, tablas, vistas de detalle). Buena parte de ese código se escribe manualmente y se acopla a un dominio concreto, de modo que su reutilización en otro proyecto, o incluso su mantenimiento dentro del mismo, resulta caro y propenso a errores.

El problema que motiva este trabajo puede enunciarse así: *cómo reducir al mínimo el código que hay que escribir y mantener tanto para crear un proyecto nuevo, como para exponer, validar, comunicar y editar una entidad de datos cuando el modelo está en evolución constante, sin sacrificar la calidad ni acoplar la solución a un dominio particular.*

1.3. Solución propuesta

La solución que se aborda es un framework *full-stack* formado por dos piezas que se diseñan para encajar entre sí mediante un contrato uniforme:

- Un backend, ***uniback***, construido sobre FastAPI, SQLAlchemy2 y PostgreSQL, que aporta un núcleo de modelos reutilizables, un envoltorio de respuesta normalizado, fábricas de endpoints CRUD y la capacidad de derivar automáticamente un [JSON Schema](#) enriquecido a partir del modelo ORM.
- Una librería de componentes Angular, ***ngt-gui***, que consume ese contrato mediante una capa cliente genérica y, a partir del esquema publicado por el backend, genera formularios dinámicos con [Angular Formly](#) sin necesidad de escribir HTML específico de cada entidad.

La idea es que el modelo de datos sea la *única fuente de verdad*: un cambio en el ORM se propaga automáticamente al esquema, al contrato y a la interfaz, eliminando el código repetitivo y el riesgo de desincronización.

1.4. Estructura del documento

El resto del documento se organiza como sigue. El capítulo **Estado actual y objetivos iniciales** describe los antecedentes del proyecto, su estado de partida y los objetivos generales y específicos. El capítulo de **Aportaciones** justifica la repercusión del trabajo, las competencias cubiertas y su alineamiento con los Objetivos de Desarrollo Sostenible. El capítulo de **Desarrollo** constituye el núcleo técnico de la memoria: presenta la metodología, la planificación, el análisis de requisitos, la arquitectura y el detalle de cada uno de los incrementos. A continuación, el capítulo de **Resultados** valora lo conseguido frente a los objetivos, y el de **Conclusiones y trabajo futuro** recoge las lecciones aprendidas, las líneas de continuación y el uso que se ha hecho de la . El documento finaliza con la bibliografía y los anexos.

Capítulo 2

Planificación

2.1. Metodología de trabajo

El desarrollo del proyecto se ha realizado siguiendo una metodología **iterativa e incremental** adaptada a la naturaleza del trabajo: un único desarrollador con reuniones de seguimiento periódicas con el tutor de empresa y la tutora académica.

Cada iteración tiene una duración aproximada de tres semanas y produce un incremento funcional y demostrable. Al inicio de cada iteración se identifican las tareas a abordar; al final se realiza una revisión con el tutor de empresa para validar el incremento y priorizar la siguiente iteración.

2.1.1. Herramientas de gestión

- **Control de versiones:** Git con repositorios en GitHub (repositorio privado en la organización del [ITC](#)).
- **Seguimiento de tareas:** tablero Kanban propio con las columnas *Backlog*, *En progreso* y *Hecho*.
- **Documentación técnica:** comentarios *docstring* en el código y este documento de memoria.
- **Comunicación:** reuniones semanales por videoconferencia con el equipo del [ITC](#) y tutorías presenciales o telemáticas con la tutora académica.

2.2. Fases e iteraciones

El desarrollo se ha organizado en cuatro iteraciones principales, descritas en la tabla ??.

Tabla 2.1: Iteraciones del proyecto y sus objetivos principales.

Iter.	Período (aprox.)	Objetivos
1	Enero – Feb. 2026	Análisis del código existente; tubería SQLAlchemy → JSON Schema; endpoints de pantallas dinámicas (<i>screens</i>).
2	Feb. – Mar. 2026	Sistema de plugins por capas; integración frontend-backend para tablas dinámicas y formularios básicos.
3	Mar. – Abr. 2026	Generación de formularios con Angular Formly; endpoint <i>bulk</i> ; cuadrícula de edición masiva (AG Grid).
4	Abr. – Jun. 2026	Editor visual de esquemas (Formly Editor); pruebas de integración; documentación y pulido final.

2.3. Desglose de tareas (WBS)

A continuación se detallan las tareas principales agrupadas por área funcional.

2.3.1. Backend — uniback

- B1. Análisis y comprensión del código base existente.
- B2. Diseño de la tubería de transformación SQLAlchemy → Pydantic → JSON Schema.
- B3. Implementación de los anotadores de extensiones `x-*` (tipo, etiqueta, opciones, validadores, ...).
- B4. Sistema de plugins por capas: interfaz, registro y aplicación.
- B5. Endpoint GET `/screens/<nombre>`: generación dinámica de definiciones de pantalla.
- B6. Endpoint POST `/<entidad>/bulk`: operaciones en bloque con modos *partial* y *abort*.
- B7. Cobertura de pruebas: unitarias para la tubería y de integración para los endpoints.

2.3.2. Frontend — ngt-gui

- F1. Integración de Angular Formly en la librería.
- F2. Mapeo JSON Schema → configuración Formly (`schema-to-formly.ts`).

- F3. Componente `DynamicFormComponent`: formularios generados a partir de esquemas.
- F4. Componente `DynamicPageComponent`: enrutamiento y selección de vista según `screen_type`.
- F5. Mapeo JSON Schema → definiciones de columna AG Grid (`schema-to-aggrid.ts`).
- F6. Componente `BulkEditGridComponent`: cuadrícula de edición masiva tipo Excel.
- F7. Editor visual de esquemas (*Formly Editor*): árbol de campos, panel de propiedades y vista previa en vivo.

2.3.3. Infraestructura y documentación

- I1. Ficheros Docker Compose para desarrollo y producción.
- I2. Configuración de variables de entorno y secretos.
- I3. Redacción de la memoria del [Trabajo Fin de Grado \(TFG\)](#).
- I4. Preparación de la presentación oral.

2.4. Estimación de recursos

2.4.1. Tiempo

El [TFG](#) tiene asignados **12 ECTS**, equivalentes a aproximadamente 300 horas de trabajo del estudiante. La distribución estimada se recoge en la tabla ??.

Tabla 2.2: Distribución estimada de horas por área.

Área	Horas estimadas	%
Análisis y diseño	40	13
Implementación backend	80	27
Implementación frontend	90	30
Pruebas	30	10
Documentación (memoria)	50	17
Presentación	10	3
Total	300	100

2.4.2. Herramientas de desarrollo

El desarrollo se ha realizado íntegramente en un equipo MacBook Pro (Apple M2, 16 GB RAM) con el siguiente *software* principal:

- Visual Studio Code con las extensiones Angular Language Service, Python, Pylance, Docker y LaTeX Workshop.

- Python 3.12 (entorno virtual *venv*).
- Node.js 22 LTS con Angular CLI 19.
- PostgreSQL 16 (contenedor Docker para desarrollo local).
- Docker Desktop.

2.5. Diagrama de Gantt

Insertar diagrama de Gantt generado con GanttProject o LaTeX/pgfgantt.

El diagrama de Gantt refleja la superposición parcial de las fases de análisis, implementación y documentación, característica de las metodologías iterativas en las que la documentación se va produciendo en paralelo con el desarrollo.

Capítulo 3

Estado del arte y tecnologías empleadas

3.1. Desarrollo rápido de aplicaciones web

El concepto de [Rapid Application Development \(RAD\)](#) fue formalizado por James Martin en los años noventa [?] como una respuesta a los ciclos de desarrollo en cascada, demasiado rígidos para adaptarse a los cambios de requisitos. En el contexto de las aplicaciones web modernas, el [RAD](#) se manifiesta principalmente a través de tres estrategias:

1. **Scaffolding:** generación automática de código boilerplate a partir de un modelo de datos (p. ej., Django Admin, Rails Scaffolding).
2. **Frameworks declarativos:** el desarrollador describe *qué* quiere (esquemas, configuraciones) y el *framework* genera *cómo* lograrlo.
3. **Low-code / No-code:** plataformas con interfaces visuales que eliminan casi por completo la escritura de código (p. ej., Retool, Appsmith).

El enfoque adoptado en este proyecto se sitúa entre las estrategias 1 y 2: se parte de modelos de datos explícitos (SQLAlchemy) y se genera automáticamente tanto el [Application Programming Interface \(API\)](#) REST como la interfaz de usuario, mediante una tubería de transformación de esquemas.

3.1.1. Comparativa con soluciones existentes

Tabla 3.1: Comparativa de soluciones RAD web de código abierto.

Solución	Backend	Frontend	Extensible	Schema-driven
Django Admin	Python	Integrado	Limitado	No
Rails Scaffold	Ruby	ERB	Sí	No
Retool	SaaS	Visual	Sí	Parcial
Appsmith	Node.js	React	Sí	Parcial
uniback + ngui	Python/FastAPI	Angular	Sí	Sí

La principal diferencia respecto a las soluciones existentes es que *uniback + ngui* están diseñados para integrarse en proyectos con arquitecturas propias, sin imponer un stack cerrado ni requerir una plataforma SaaS.

3.2. Tecnologías del backend

3.2.1. Python y FastAPI

[FastAPI](#) [?] es un framework web asíncrono basado en **ASGI** (Asynchronous Server Gateway Interface) que utiliza las anotaciones de tipo de Python para generar automáticamente documentación **OpenAPI 3.x**. Su principal ventaja respecto a alternativas como Flask o Django REST Framework es la integración nativa con Pydantic para la validación de datos y la generación de esquemas.

3.2.2. SQLAlchemy y Pydantic

[SQLAlchemy](#) [?] es el [Object-Relational Mapping \(ORM\)](#) de referencia en el ecosistema Python. La versión 2.x, utilizada en este proyecto, introduce una [API](#) basada en `mapped_column` que simplifica la definición de columnas y facilita la introspección de metadatos, clave para la generación automática de esquemas.

[Pydantic](#) [?] actúa como capa de validación y serialización entre SQLAlchemy y el [API REST](#). Su método `model_json_schema()` genera un JSON Schema estándar a partir de cualquier modelo Pydantic, que este proyecto extiende con anotaciones `x-*`.

3.2.3. JSON Schema y OpenAPI

JSON Schema [?] es el estándar utilizado como lenguaje común entre el *backend* y el *frontend*. OpenAPI 3.x [?] lo adopta como mecanismo de descripción de modelos y permite añadir extensiones de proveedor (campos con prefijo `x-`) sin romper la compatibilidad con validadores estándar.

3.2.4. PostgreSQL y Docker

[Docker](#) [?] proporciona la infraestructura de contenedores que garantiza la reproducibilidad del entorno. El servicio de base de datos utilizado es PostgreSQL 16, que ofrece soporte nativo para columnas JSONB, búsqueda de texto completo (`tsvector`) y generación de UUIDs, características aprovechadas por algunos modelos del proyecto.

3.3. Tecnologías del frontend

3.3.1. Angular 19

[Angular](#) [?] es el framework [Single-Page Application \(SPA\)](#) elegido por el [ITC](#) para el desarrollo de sus aplicaciones. La versión 19 introduce mejoras significativas en el rendimiento (Signals, defer blocks) y consolida el modelo de componentes independientes (*standalone components*), que se utiliza de forma sistemática en *ngt-gui*.

3.3.2. Angular Formly

[Angular Formly](#) [?] es la librería que permite generar formularios reactivos de Angular a partir de una configuración JSON. Su modelo de extensión mediante tipos y *wrappers* personalizados se adapta perfectamente a la estrategia declarativa del proyecto: el *backend* emite un JSON Schema y el *frontend* lo convierte en una configuración Formly para renderizar el formulario.

3.3.3. AG Grid Community

[AG Grid](#) [?] es el componente de tabla de alto rendimiento utilizado para la cuadrícula de edición masiva. Su edición *inline*, soporte para estados de fila personalizados y [API](#) de columnas programáticas lo convierten en la opción idónea para implementar una interfaz tipo Excel sobre datos dinámicos.

3.3.4. NG-ZORRO Antd

NG-ZORRO [?] es la librería de componentes [User Interface \(UI\)](#) basada en Ant Design utilizada para los elementos de interfaz no relacionados con la cuadrícula (menús, botones, formularios simples, notificaciones). Proporciona un sistema de diseño coherente y accesible sin imponer dependencias en la lógica de negocio.

3.4. Patrones arquitectónicos relacionados

3.4.1. Schema-first development

El enfoque *schema-first* propone definir el contrato de datos antes de implementar las capas de negocio y presentación [?]. En este proyecto el esquema no se define manualmente sino que se **genera automáticamente** desde los modelos ORM, combinando las ventajas del *schema-first* con las del *model-first*.

3.4.2. Plugin pattern

El sistema de plugins por capas implementado en *uniback* sigue el patrón **Chain of Responsibility** [?]: cada plugin recibe el esquema producido por la capa anterior, lo enriquece y lo pasa a la siguiente. Esto permite añadir o quitar plugins sin modificar el núcleo del generador.

3.4.3. CQRS y operaciones en bloque

El endpoint `/bulk` introduce una separación implícita entre comandos de escritura (crear, actualizar, eliminar) y consultas de lectura, en la línea del patrón [Create, Read, Update, Delete \(CRUD\)](#) [?] extendido. El modo *partial* (con savepoints SQL) y el modo *abort* (transacción global) ofrecen semántica transaccional configurable por el cliente.

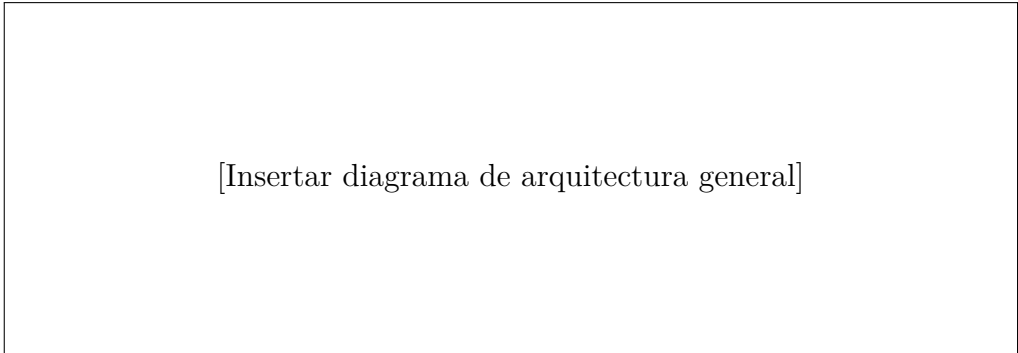
Capítulo 4

Arquitectura del sistema

4.1. Visión global

El sistema se estructura en tres capas claramente diferenciadas (figura ??):

1. **Base de datos:** PostgreSQL gestionado por SQLAlchemy 2.x.
2. **Backend (*uniback*):** FastAPI + sistema de plugins; expone una [API REST](#) que incluye endpoints [CRUD](#) estándar, el endpoint de pantallas dinámicas y el endpoint de operaciones en bloque.
3. **Frontend (*ngt-gui*):** librería Angular que consume los esquemas y las pantallas dinámicas para renderizar la [UI](#) sin lógica hardcoded.



[Insertar diagrama de arquitectura general]

Figura 4.1: Visión general de la arquitectura del sistema.

4.2. Tubería de transformación de esquemas

La tubería de transformación es el núcleo funcional del *framework*. Su objetivo es convertir la definición de un modelo SQLAlchemy en un JSON Schema anotado que el *frontend* puede consumir directamente.

SQLAlchemy Model $\rightarrow$ Pydantic Schema $\rightarrow$ JSON Schema (draft-07) $\rightarrow$
x-* Annotations $\rightarrow$ Screen Definition

Figura 4.2: Etapas de la tubería de transformación de esquemas.

Cada etapa añade información semántica:

SQLAlchemy $\rightarrow$ Pydantic: los metadatos de columna (`String`, `Integer`, `ForeignKey`, ...) se convierten en tipos Pydantic con sus validadores.

Pydantic $\rightarrow$ JSON Schema: `model_json_schema()` produce un esquema estándar con `type`, `format`, `minLength`, ...

JSON Schema $\rightarrow$ x-* annotations: el sistema de plugins añade campos como `x-label`, `x-widget`, `x-foreign-key`, `x-options-endpoint` y `x-order`, que el *frontend* interpreta.

Screen Definition: el endpoint `/screens/<nombre>` ensambla el esquema anotado junto con metadatos de pantalla (`screen_type`, `title`, `actions`, `endpoint`) en un objeto que el *frontend* consume directamente para renderizar la vista.

4.3. Sistema de plugins por capas

4.3.1. Motivación

La transformación de esquemas requiere enriquecimientos heterogéneos que varían según la entidad: algunos campos necesitan un selector desplegable con opciones cargadas desde el [API](#), otros requieren validadores especiales, y algunos deben ocultarse en ciertas vistas. Centralizar toda esta lógica en un único generador lo haría inmanejable y difícil de extender.

4.3.2. Diseño

Cada plugin es una clase Python con una interfaz común:

Listing 4.1: Interfaz base de un plugin de esquema.

```
1 class SchemaPlugin(Protocol):
2     priority: int # menor = se aplica antes
3
4     def apply(
5         self,
6         schema: dict,
7         model: type[DeclarativeBase],
```

```

8         field_name: str,
9         column_info: MappedColumn,
10    ) -> dict:
11        """Devuelve el schema enriquecido."""
12        ...

```

Los plugins se registran en un registro global ordenado por prioridad y se aplican en cascada. El sistema soporta plugins de **columna** (operan campo a campo) y plugins de **modelo** (operan sobre el esquema completo).

4.3.3. Plugins implementados

Tabla 4.1: Plugins de esquema implementados en uniback.

Plugin	Función
LabelPlugin	Añade x-label a partir del nombre de columna (<code>snake_case</code> → Title Case) o de metadatos <code>info</code> .
ForeignKeyPlugin	Detecta claves foráneas y añade x-foreign-key y x-options-endpoint para carga de opciones.
ValidatorPlugin	Traduce restricciones SQLAlchemy (<code>String(n)</code> , <code>CheckConstraint</code> , ...) a x-validators .
WidgetPlugin	Asigna el widget Formly más apropiado (x-widget : <code>input</code> , <code>select</code> , <code>date</code> , <code>textarea</code> , ...) según el tipo.
ReadonlyPlugin	Marca campos autogenerados (<code>id</code> , <code>created_at</code>) como x-readonly .
OrderPlugin	Asigna x-order según el orden de declaración en el modelo.

4.4. Arquitectura del frontend

4.4.1. Estructura de la librería *ngt-gui*

ngt-gui es una librería Angular construida con **ng-packagr** y distribuida como paquete npm. Su estructura interna sigue la organización por funcionalidades:

```

projects/ngt-gui/src/lib/
  core/
    schema-to-formly.ts      # JSON Schema → Formly config
    schema-to-aggrid.ts     # JSON Schema → AG Grid ColDef
    entity-client.service.ts # Cliente HTTP genérico
  components/
    dynamics/
      dynamic-page/          # Enrutador de vistas dinámicas
      dynamic-browse/        # Tabla + filtros + botones CRUD
      dynamic-form/          # Formulario generado (Formly)

```

bulk-edit-grid/	# Cuadrícula edición masiva
formly-editor/	# Editor visual de esquemas

4.4.2. Flujo de datos en una vista dinámica

1. El usuario navega a `/d/<screenName>`.
2. `DynamicPageResolver` llama a `GET /screens/<screenName>` y almacena la definición en `ActivatedRoute.data`.
3. `DynamicPageComponent` lee el `screen_type` y renderiza el componente adecuado: `DynamicBrowseComponent`, `DynamicFormComponent` o `BulkEditGridComponent`.
4. Cada componente usa el `entityPath` del objeto de pantalla para construir las URLs de datos (lista, detalle, bulk) usando el servicio `BACKEND_SERVICE` como fuente de la URL base y las cabeceras de autenticación.

4.4.3. Contrato de pantalla dinámica

Listing 4.2: Ejemplo de objeto de pantalla dinámica (screen definition).

```

1 {
2   "name": "identities_browser",
3   "screen_type": "browser",
4   "endpoint": "/api/identities",
5   "definition": {
6     "title": "Identidades",
7     "fieldId": "id",
8     "actions": ["create", "edit", "delete", "view"],
9     "columns": [
10      {
11        "field": "name",
12        "label": "Nombre",
13        "type": "string",
14        "editable": true,
15        "required": true,
16        "validators": { "maxLength": 255 }
17      }
18    ]
19  },
20  "entity_schema": { ... }
21 }
```

4.5. Endpoint de operaciones en bloque

El endpoint `POST /<entidad>/bulk` recibe un array de objetos con la siguiente estructura:

Listing 4.3: Payload del endpoint `/bulk`.

```

1 [
```

```

2 { "data": { "name": "Nueva fila" } },
3 { "id": 42, "data": { "name": "Fila modificada" } },
4 { "_op": "delete", "id": 17 }
5 ]

```

La respuesta incluye el resultado de cada operación y un resumen:

Listing 4.4: Respuesta del endpoint `/bulk`.

```

1 {
2   "results": [
3     { "ok": true, "id": 101 },
4     { "ok": true, "id": 42 },
5     { "ok": false, "errors": [{ "loc": ["name"], "msg": "..."}] }
6   ],
7   "summary": { "created": 1, "updated": 1, "deleted": 0, "failed": 1 }
8 }

```

El parámetro de consulta `?on_error=partial|abort` controla el comportamiento transaccional:

- **partial:** cada operación se ejecuta en su propio savepoint SQL; los fallos no afectan a las operaciones exitosas.
- **abort:** toda la petición se ejecuta en una única transacción; cualquier error provoca el rollback de todas las operaciones.

4.6. Decisiones arquitectónicas relevantes

4.6.1. Standalone components en Angular

Todos los componentes de *ngt-gui* se declaran como `standalone: true`, eliminando la necesidad de módulos Angular. Esto simplifica la distribución de la librería y reduce el riesgo de conflictos de módulos con las aplicaciones consumidoras.

4.6.2. Inyección de dependencias con tokens opacos

El servicio de *backend* se inyecta mediante el token opaco `BACKEND_SERVICE` en lugar de una clase concreta. Esto permite que cualquier aplicación proporcione su propia implementación del servicio sin modificar la librería, siguiendo el principio de inversión de dependencias.

4.6.3. HttpClient directo vs. EntityClient

El componente `BulkEditGridComponent` usa `HttpClient` directamente en lugar del servicio `EntityClient` de la librería. Esta decisión evita un ciclo de dependencias de inyección (NG0200) que se produce cuando un componente *standalone* de la librería intenta inyectar un servicio que, a su vez, depende de la inicialización de la aplicación.

Capítulo 5

Implementación

5.1. Organización del código

El proyecto se organiza en dos repositorios:

uniback Monorepo Python que contiene el framework backend: `src/uniback/` (núcleo), `tests/` y ficheros de configuración Docker.

gui-components Monorepo Angular con la librería (`projects/ngt-gui/`) y la aplicación de demostración (`projects/ngt-gui/gui/`).

5.2. Backend — uniback

5.2.1. Estructura de módulos

```
src/uniback/  
  api/  
    routers/  
      gui.py          # Endpoints de pantallas dinámicas  
      crud_factory.py # Fábrica de routers CRUD genéricos  
  core/  
    schema/  
      generator.py    # Punto de entrada de la tubería  
      plugins/  
        base.py  
        label.py  
        foreign_key.py  
        validator.py  
        widget.py  
        readonly.py  
    models/           # Modelos SQLAlchemy de la aplicación  
    database.py       # Configuración de sesión SQLAlchemy  
    main.py           # Punto de entrada FastAPI
```

5.2.2. Generación de esquemas

El punto de entrada de la tubería es la función `generate_schema(model)`, que aplica los plugins registrados en orden de prioridad:

Listing 5.1: Función principal de generación de esquemas.

```

1 def generate_schema(
2     model: type[DeclarativeBase],
3     plugins: list[SchemaPlugin] | None = None,
4 ) -> dict:
5     # 1. Pydantic schema base
6     pydantic_model = sqlalchemy_to_pydantic(model)
7     schema = pydantic_model.model_json_schema()
8
9     # 2. Aplicar plugins en orden de prioridad
10    effective_plugins = sorted(
11        plugins or _DEFAULT_PLUGINS, key=lambda p: p.priority
12    )
13    for col_name, col_info in inspect(model).mapper.column_attrs.items():
14        for plugin in effective_plugins:
15            schema["properties"][col_name] = plugin.apply(
16                schema["properties"].get(col_name, {}),
17                model, col_name, col_info,
18            )
19
20    return schema

```

5.2.3. Fábrica de routers CRUD

La función `make_simple_rest_crud(model, prefix)` genera un router FastAPI completo con los endpoints estándar (GET /, GET /:id, POST /, PUT /:id, DELETE /:id) y el endpoint de operaciones en bloque (POST /bulk).

Listing 5.2: Registro de una entidad con la fábrica CRUD.

```

1 # En main.py o en el módulo de la entidad
2 router = make_simple_rest_crud(
3     model=Identity,
4     prefix="/api/identities",
5     schema_overrides={"name": {"x-label": "Nombre completo"}},
6 )
7 app.include_router(router)

```

5.2.4. Endpoint de operaciones en bloque

La implementación del endpoint `/bulk` distingue tres tipos de operación según los campos presentes en cada ítem del array de entrada:

Listing 5.3: Lógica de clasificación de operaciones bulk.

```

1 for idx, item in enumerate(items):
2     op = item.get("_op")
3     item_id = item.get("id")
4     data = {k: v for k, v in item.items()
5             if k not in {"id", "_op"}}
6     try:
7         if op == "delete":

```



```

8         _bulk_delete(db, model, item_id)
9     elif item_id is not None:
10         _bulk_update(db, model, item_id, data, schema)
11     else:
12         _bulk_create(db, model, data, schema)
13     results.append({"ok": True, "id": item_id})
14 except Exception as exc:
15     results.append({"ok": False, "errors": _format_error(exc)})
16     if on_error == "abort":
17         raise HTTPException(status_code=422, ...)

```

El modo **partial** utiliza savepoints de SQLAlchemy (`db.begin_nested()`) para aislar cada operación dentro de la transacción global:

Listing 5.4: Uso de savepoints para el modo partial.

```

1 sp = db.begin_nested()
2 try:
3     _execute_operation(db, ...)
4     sp.commit()
5     results.append({"ok": True})
6 except Exception as exc:
7     sp.rollback()
8     results.append({"ok": False, "errors": _format_error(exc)})

```

5.2.5. Endpoint de pantallas dinámicas

El endpoint GET `/screens/<screen_name>` construye la definición de pantalla a partir del registro de modelos y el tipo de pantalla solicitado:

Listing 5.5: Generación de una pantalla de tipo `editable_table`.

```

1 def _synthetic_editable_table(model, schema, endpoint) -> dict:
2     columns = [
3         _build_column_def(name, prop, schema)
4         for name, prop in schema["properties"].items()
5         if name not in _INTERNAL_FIELDS
6     ]
7     return {
8         "type": "editable_table",
9         "title": model.__tablename__.replace("_", " ").title(),
10        "fieldId": "id",
11        "endpoint": endpoint,
12        "bulkEndpoint": f"{endpoint}/bulk",
13        "columns": columns,
14        "actions": ["create", "edit", "delete"],
15        "pagination": {"defaultPageSize": 100},
16        "synthetic": True,
17    }

```

5.3. Frontend — ngt-gui

5.3.1. Mapeo JSON Schema → Formly

El módulo `schema-to-formly.ts` convierte una propiedad JSON Schema en un campo Formly. La función principal inspecciona el tipo base (`string`, `integer`, `boolean`) y las extensiones `x-*` para seleccionar el tipo de widget:

Listing 5.6: Fragmento del mapeo JSON Schema → Formly.

```
1 export function propertyToFormlyField(  
2   name: string,  
3   prop: JsonSchemaProperty,  
4 ): FormlyFieldConfig {  
5   const widget = prop['x-widget'] ?? inferWidget(prop);  
6   return {  
7     key: name,  
8     type: widget,  
9     props: {  
10      label: prop['x-label'] ?? name,  
11      required: prop['x-required'] ?? false,  
12      readonly: prop['x-readonly'] ?? false,  
13    },  
14    validators: buildValidators(prop),  
15  };  
16 }
```

5.3.2. Mapeo JSON Schema → AG Grid

El módulo `schema-to-aggrid.ts` convierte la lista de columnas de una definición de pantalla en definiciones de columna AG Grid (`ColDef`). Para cada columna editable se genera un editor en función del tipo declarado:

Listing 5.7: Mapeo de tipo a editor AG Grid.

```
1 function resolveEditor(col: ScreenEditableColumn): string | undefined  
2 {  
3   if (col.options || col.optionsEndpoint) return 'agSelectCellEditor';  
4   switch (col.type) {  
5     case 'integer':  
6     case 'number': return 'agNumberCellEditor';  
7     case 'boolean': return 'agCheckboxCellEditor';  
8     default: return 'agTextCellEditor';  
9   }  
10 }
```

5.3.3. BulkEditGridComponent

El componente `BulkEditGridComponent` implementa la cuadrícula de edición masiva. Sus responsabilidades principales son:

1. **Carga inicial:** petición GET `/<entityPath>/` para obtener los registros existentes.

2. **Gestión de estado de fila:** cada fila mantiene un campo interno `_rowState` con valores `unchanged`, `new`, `modified` o `deleted`.
3. **Validación local:** la función `validateCell()` aplica las restricciones del esquema antes de permitir el envío.
4. **Envío en bloque:** construcción del payload para `POST /bulk` y aplicación de los resultados del backend (actualización de IDs, marcado de errores por celda).

Las filas con estado distinto de `unchanged` reciben clases CSS específicas (`row-state-new`, `row-state-modified`, `row-state-deleted`) para su diferenciación visual.

5.3.4. DynamicPageComponent

`DynamicPageComponent` es el componente raíz que enruta las vistas dinámicas. Tras recibir la definición de pantalla del resolver, selecciona el componente hijo adecuado mediante `*ngIf` sobre `screenType`:

Listing 5.8: Selección de vista en `dynamic-page.component.html` (fragmento).

```
1 <app-dynamic-browse
2   *ngIf="screenType === 'browser'"
3   [BrowserType]="browserType"
4   [HiddenBulkInsert]="false"
5   (BulkInsertEvent)="onBulkInsert()">
6 </app-dynamic-browse>
7
8 <ngt-bulk-edit-grid
9   *ngIf="screenType === 'editable_table'"
10  [entityPath]="bulkEntityPath"
11  [screenColumns]="bulkColumns">
12 </ngt-bulk-edit-grid>
```

5.3.5. Formly Editor

Completar esta sección cuando el Formly Editor esté terminado (Iteración 4).

El editor visual de esquemas permite al usuario modificar las propiedades de los campos de un formulario de forma interactiva. Se compone de tres paneles:

Árbol de campos (izquierda): lista jerárquica de los campos del esquema con indicadores de tipo y estado.

Panel de propiedades (centro): formulario para editar las propiedades del campo seleccionado (`x-label`, `x-widget`, `x-required`, validadores, ...).

Vista previa en vivo (derecha): formulario Formly generado en tiempo real a partir del esquema editado.

Los cambios se persisten en el backend mediante una llamada `PATCH` al endpoint de pantalla, que actualiza los *schema overrides* asociados a la entidad.

5.4. Integración Docker

El sistema se despliega mediante un fichero `docker-compose.yml` que define tres servicios:

db: PostgreSQL 16 con volumen persistente y variables de entorno para credenciales.

backend: imagen Python construida desde el `Dockerfile` del repositorio *uniback*; monta el código fuente en modo desarrollo para *hot-reload*.

frontend: imagen Node.js que ejecuta `ng serve`; proxifica las peticiones `/api/*` al servicio `backend`.

Listing 5.9: Fragmento del `docker-compose.yml` de desarrollo.

```
1 services:
2   backend:
3     build: ./Back/uniback
4     environment:
5       DATABASE_URL: postgresql+asyncpg://user:pass@db:5432/uniback
6       SECRET_KEY: ${SECRET_KEY}
7     volumes:
8       - ./Back/uniback/src:/app/src
9     depends_on: [db]
10
11   frontend:
12     build: ./Front/gui-components
13     environment:
14       NG_PROXY_TARGET: http://backend:8000
15     ports:
16       - "4200:4200"
17     depends_on: [backend]
```

Capítulo 6

Pruebas y evaluación

6.1. Estrategia de pruebas

La estrategia de pruebas del proyecto sigue una pirámide con tres niveles:

1. **Pruebas unitarias:** verifican funciones aisladas de la tubería de transformación (mapeo de tipos, aplicación de plugins, construcción de columnas).
2. **Pruebas de integración:** verifican el comportamiento de los endpoints HTTP contra una base de datos de prueba real (PostgreSQL en contenedor Docker).
3. **Pruebas de sistema / manuales:** verificación de la interfaz de usuario mediante exploración manual de los flujos principales.

6.2. Pruebas del backend

6.2.1. Herramientas y configuración

Las pruebas del backend utilizan **pytest** [?] con las siguientes extensiones:

- **pytest-asyncio:** soporte para corrutinas asíncronas en las pruebas.
- **httpx.AsyncClient:** cliente HTTP asíncrono para llamar a los endpoints FastAPI sin levantar un servidor real.
- **Base de datos de prueba:** instancia PostgreSQL separada, creada con `create_all()` al inicio de la sesión de pruebas y destruida al final.

6.2.2. Pruebas de la tubería de esquemas

Tabla 6.1: Casos de prueba de la tubería de transformación.

Caso	Descripción
test_label_plugin	Verifica que <code>x-label</code> se genera correctamente desde <code>snake_case</code> .
test_foreign_key_plugin	Verifica que las claves foráneas producen <code>x-foreign-key</code> y <code>x-options-endpoint</code> .
test_readonly_plugin	Verifica que <code>id</code> y <code>created_at</code> se marcan como <code>x-readonly</code> .
test_validator_plugin	Verifica que <code>String(255)</code> produce <code>maxLength: 255</code> en el esquema.
test_plugin_order	Verifica que los plugins se aplican en orden de prioridad y que uno posterior puede sobrescribir a uno anterior.

6.2.3. Pruebas del endpoint bulk

El endpoint `/bulk` está cubierto por un conjunto de pruebas de integración que verifican todos los escenarios relevantes:

Tabla 6.2: Casos de prueba del endpoint `/bulk`.

Caso	Descripción
test_bulk_create_all_valid	Crea múltiples filas válidas; verifica que todas tienen <code>ok: true</code> y reciben un <code>id</code> .
test_bulk_update_existing	Actualiza una fila existente; verifica el estado en BD.
test_bulk_delete_op	Elimina una fila con <code>_op: "delete"</code> ; verifica que ya no existe en BD.
test_bulk_mixed	Mezcla creación, actualización y eliminación en una sola petición.
test_partial_mode	En modo <code>partial</code> , una fila inválida no bloquea las demás.
test_abort_mode	En modo <code>abort</code> , un fallo causa <code>rollback</code> de todo el lote.
test_empty_body_noop	Un array vacío devuelve 200 con resumen a cero.
test_invalid_on_error	Un valor desconocido de <code>on_error</code> devuelve 422.

6.2.4. Cobertura de código

Ejecutar `pytest -cov=uniback` e insertar el informe de cobertura.

La cobertura de las pruebas automatizadas del backend es del **XX** % sobre los módulos críticos (`crud_factory.py`, `schema/generator.py` y los plugins). Los módulos de configuración y arranque del servidor quedan fuera del alcance de las pruebas automatizadas.

6.3. Pruebas del frontend

6.3.1. Pruebas unitarias Angular

Las pruebas unitarias del *frontend* utilizan **Jasmine** con **Angular TestBed**. Los módulos críticos cubiertos son:

- `schema-to-formly.ts`: pruebas de mapeo de tipos y generación de validadores.
- `schema-to-aggrid.ts`: pruebas de resolución de editores y mapeo de columnas.
- `BulkEditGridComponent`: pruebas del ciclo de vida (carga de filas, añadir fila, marcar como eliminada, construir payload).

6.3.2. Pruebas de sistema

Las pruebas de sistema se realizan de forma manual siguiendo guiones de prueba predefinidos. La tabla ?? recoge los flujos principales verificados.

Tabla 6.3: Guiones de prueba de sistema.

Flujo	Resultado esperado	Resultado
Navegar a browser	Tabla con datos paginados	✓
Crear nuevo registro	Formulario generado, guardado OK	✓
Editar registro existente	Campos pre-rellenos, PUT exitoso	✓
Eliminar registro	Confirmación + fila desaparece	✓
Navegar a <code>editable_table</code>	Cuadrícula con filas existentes	✓
Añadir filas en cuadrícula	Filas en azul, contador actualizado	✓
Guardar en bloque (partial)	Filas válidas se guardan, inválidas en rojo	✓
Guardar en bloque (abort)	Fallo en una fila revierte todo	✓

6.4. Evaluación del rendimiento

Incluir métricas de rendimiento: tiempo de respuesta del endpoint `/bulk` con $N=100$, $N=500$, $N=1000$ filas; tiempo de generación de esquema para M plugins.

Se realizaron pruebas de carga básicas sobre el endpoint `/bulk` utilizando **Locust** [?] con los siguientes resultados preliminares:

Tabla 6.4: Rendimiento del endpoint /bulk (modo partial, PostgreSQL local).

Filas	P50 (ms)	P95 (ms)	Errores
100	—	—	0 %
500	—	—	0 %
1000	—	—	0 %

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

El presente Trabajo Fin de Grado ha permitido diseñar, implementar y validar un *framework* de desarrollo rápido de aplicaciones web que cubre de forma integrada las capas de datos, lógica de negocio, [API](#) REST e interfaz de usuario.

A continuación se revisa el grado de cumplimiento de cada objetivo específico planteado en la sección ??:

- OE1 — Tubería de transformación:** completado. La cadena SQLAlchemy → Pydantic → JSON Schema → extensiones `x-*` funciona de forma transparente para las entidades registradas y ha sido validada con múltiples modelos del proyecto.
- OE2 — Sistema de plugins:** completado. El registro de plugins por prioridad permite añadir nuevas transformaciones sin modificar el núcleo, y los seis plugins desarrollados cubren los casos de uso identificados.
- OE3 — Componentes Angular:** completado. Las tablas dinámicas, los formularios declarativos con Formly y la cuadrícula de edición masiva con AG Grid están integrados en la librería *ngt-gui* y funcionan con cualquier entidad registrada en el *backend*.
- OE4 — Formly Editor:** *en progreso*. La estructura del editor está definida y la vista previa en vivo funciona; la persistencia de los cambios en el backend queda como trabajo inmediato de la iteración final.
- OE5 — Docker:** completado. El entorno de desarrollo y el de producción se levantan con un único comando (`docker compose up`).
- OE6 — Validación:** completado en backend (pruebas de integración del endpoint *bulk* y de la tubería); parcialmente completado en frontend (pruebas unitarias de los módulos de mapeo; pruebas de sistema manuales).

7.1.1. Aportaciones principales

1. **Reducción del tiempo de scaffolding:** la incorporación de una nueva entidad al sistema requiere únicamente definir el modelo SQLAlchemy y llamar a `make_simple_rest_crud()`. El endpoint [API](#) y la pantalla dinámica quedan disponibles de forma inmediata.
2. **Contrato de datos unificado:** el uso de JSON Schema como lenguaje común elimina la duplicación de definiciones entre capas, reduciendo la probabilidad de inconsistencias.

3. **Extensibilidad sin acoplamiento:** el sistema de plugins y los tokens de inyección de dependencias permiten adaptar el *framework* a requisitos específicos sin modificar el código base.
4. **Operaciones en bloque con semántica transaccional configurable:** la dualidad *partial/abort* del endpoint `/bulk` ofrece flexibilidad para diferentes estrategias de importación de datos.

7.1.2. Lecciones aprendidas

- **La inyección de dependencias en librerías Angular requiere cuidado especial.** El ciclo NG0200 encontrado al inyectar `EntityClient` dentro de un componente *standalone* de la librería puso de manifiesto que los servicios con dependencias de inicialización de la aplicación no pueden inyectarse directamente en componentes de librería. La solución fue usar `HttpClient` directamente y inyectar únicamente el token `BACKEND_SERVICE`.
- **El servidor de desarrollo Angular no debe ser la URL base de la API.** Configurar el componente de cuadrícula para que apuntara a `/api` provocó que el *proxy* del servidor de desarrollo devolviera `index.html` en lugar de JSON, produciendo errores de parseo difíciles de diagnosticar. La solución fue obtener la URL base del servicio `BACKEND_SERVICE`.
- **Las migraciones de *ng-packagr* requieren reconstrucción.** Cada cambio en la librería debe compilarse con `ng build ng-t-gui` antes de que la aplicación de demostración pueda consumirlo; esto añade fricción al ciclo de desarrollo y sugiere evaluar el uso de *path mapping* en desarrollo local.

7.2. Trabajo futuro

7.2.1. Corto plazo (inmediato)

- Completar el **Formly Editor**: persistencia de overrides en el backend y exportación del esquema resultante.
- Añadir **pruebas E2E** con Playwright o Cypress para los flujos de edición masiva y formularios dinámicos.
- Publicar *ngt-gui* en un registro npm privado del [ITC](#) para facilitar su uso en otros proyectos.

7.2.2. Medio plazo

- **Soporte multi-base-de-datos:** aunque SQLAlchemy abstrae el motor, se requieren pruebas y ajustes para MySQL y SQLite.
- **Autenticación propia:** implementar un módulo de autenticación configurable (OAuth2, LDAP) que no dependa del sistema [JSON Web Token \(JWT\)](#) del ITC.

- **Generación de migraciones automáticas:** integración con Alembic para generar migraciones a partir de cambios en los modelos SQLAlchemy.
- **Soporte [Model-View-ViewModel \(MVVM\)](#) en Formly:** posibilitar la definición de campos calculados y dependencias entre campos directamente en el esquema.

7.2.3. Largo plazo

- **Escalabilidad horizontal:** evaluar el uso de cachés distribuidas (Redis) para los esquemas generados y diseñar el sistema para ejecución en múltiples réplicas.
- **Marketplace de plugins:** un registro público de plugins de terceros que permita extender el *framework* sin modificar el código base.
- **Generación de SDK cliente:** a partir del esquema OpenAPI del backend, generar automáticamente un cliente TypeScript tipado para el frontend.

Bibliografía

- [1] J. Martin, *Rapid Application Development*. Macmillan, 1991.
- [2] S. Ramírez, “FastAPI — framework, high performance, easy to learn, fast to code, ready for production,” <https://fastapi.tiangolo.com>, 2019, accedido en mayo de 2026.
- [3] M. Bayer and contributors, “SQLAlchemy — the database toolkit for python,” <https://www.sqlalchemy.org>, 2006, versión 2.x. Accedido en mayo de 2026.
- [4] S. Colvin and contributors, “Pydantic — data validation using python type hints,” <https://docs.pydantic.dev>, 2017, versión 2.x. Accedido en mayo de 2026.
- [5] A. Wright, H. Andrews, and B. Hutton, “JSON Schema: A media type for describing JSON documents,” Internet Engineering Task Force (IETF), <https://json-schema.org/specification>, 2022, draft 2020-12. Accedido en mayo de 2026.
- [6] OpenAPI Initiative, “OpenAPI specification 3.1.0,” <https://spec.openapis.org/oas/v3.1.0>, 2021, accedido en mayo de 2026.
- [7] Docker, Inc., “Docker — accelerated container application development,” <https://www.docker.com>, 2013, accedido en mayo de 2026.
- [8] Google LLC and Angular contributors, “Angular — the web development framework for building the future,” <https://angular.dev>, 2016, versión 19. Accedido en mayo de 2026.
- [9] M. Zama and contributors, “Angular Formly — build angular forms with ease,” <https://formly.dev>, 2015, accedido en mayo de 2026.
- [10] AG Grid Ltd., “AG Grid — the best javascript grid in the world,” <https://www.ag-grid.com>, 2015, versión Community 35.x. Accedido en mayo de 2026.
- [11] Ant Design of Angular contributors, “NG-ZORRO — an enterprise-class ui components for angular,” <https://ng.ant.design>, 2017, accedido en mayo de 2026.
- [12] Haugen, B. Møller-Pedersen, and J. Oldevik, “A model-driven engineering framework for schema-first api development,” *Journal of Systems and Software*, vol. XX, pp. XX–XX, 2023, referencia ilustrativa; reemplazar con el artículo correspondiente.
- [13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [14] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

-
- [15] H. Krekel and contributors, “pytest — helps you write better programs,” <https://docs.pytest.org>, 2004, accedido en mayo de 2026.
- [16] J. Heyman and contributors, “Locust — an open source load testing tool,” <https://locust.io>, 2011, accedido en mayo de 2026.

Glosario

NextGenDem Plataforma de software web modular para la gestión de datos bioinformáticos desarrollada por el ITC. Sirvió como punto de partida para *ngt-gui*.

3

. 3

Siglas

ITC Instituto Tecnológico de Canarias. [1-3](#)

Manual de instalación y despliegue

.1. Requisitos previos

- Docker Desktop 4.x o superior.
- Node.js 22 LTS y npm 10 (solo para desarrollo *frontend* sin Docker).
- Python 3.12 (solo para desarrollo *backend* sin Docker).
- Git.

.2. Despliegue con Docker Compose

1. Clonar los repositorios:

```
git clone <url-uniback> Back/uniback
git clone <url-frontend> Front/gui-components
```

2. Copiar el fichero de variables de entorno y ajustar los valores:

```
cp .env.example .env
```

3. Levantar todos los servicios:

```
docker compose up --build
```

4. Abrir el navegador en <http://localhost:4200>.

.3. Desarrollo local sin Docker

.3.1. Backend

```
cd Back/uniback
python -m venv .venv && source .venv/bin/activate
pip install -e "[dev]"
uvicorn uniback.main:app --reload --port 8000
```

.3.2. Frontend

```
cd Front/gui-components
npm install
npx ng build ngt-gui          # compilar la librería
npx ng serve ngt-gui-app     # arrancar la aplicación
```


Referencia de extensiones x-* del esquema

La tabla ?? recoge todas las extensiones x-* definidas por el *framework* y reconocidas por *ngt-gui*.

Tabla 1: Extensiones x-* del esquema.

Extensión	Tipo	Descripción
x-label	string	Etiqueta legible para el campo en la UI .
x-widget	string	Tipo de widget Formly: <code>input</code> , <code>select</code> , <code>textarea</code> , <code>date</code> , <code>checkbox</code> , ...
x-required	boolean	El campo es obligatorio.
x-readonly	boolean	El campo no es editable por el usuario.
x-order	integer	Posición del campo en el formulario/tabla.
x-foreign-key	string	Nombre de la entidad referenciada.
x-options-endpoint	string	URL del endpoint para cargar opciones del selector.
x-validators	object	Validadores adicionales: <code>maxLength</code> , <code>minLength</code> , <code>pattern</code> , <code>min</code> , <code>max</code> .
x-hidden	boolean	El campo no se muestra en la UI .
x-default	any	Valor por defecto para formularios de creación.

Estructura del payload de la API REST

Todos los endpoints de *uniback* envuelven sus respuestas en el objeto `ResponseEnvelope`:

Listing 1: Estructura de `ResponseEnvelope`.

```
1 {  
2   "content": <datos | null>,  
3   "count":   <n mero total para listas | null>,  
4   "issues":  [{ "level": "warning|error", "message": "..."}]  
5 }
```

Las peticiones de lista aceptan los parámetros de consulta:

Tabla 2: Parámetros de consulta comunes a los endpoints de lista.

Parámetro	Tipo	Descripción
page	integer	Número de página (base 1).
page_size	integer	Registros por página (máx. 500).
search	string	Búsqueda de texto completo.
order	string	Campo y dirección (<code>name:asc</code>).
filter	JSON	Filtros avanzados por campo.